A Deep Dive into Rainbow DQN

Chapter 2: Prioritized Experience Replay (PER)

Harsh Verma

June 26, 2025

Abstract

Standard Experience Replay in DQN samples past transitions uniformly, treating all experiences as equally important. This is inefficient, as many experiences offer little new information. Prioritized Experience Replay (PER), proposed by Schaul et al. (2015), addresses this by replaying "important" transitions more frequently, enabling the agent to learn more effectively from its most surprising experiences. This chapter provides a first-principles derivation of PER, detailing the motivation, the core mechanism, the necessary data structures for efficient implementation, and its synergistic role within the Rainbow agent.

1 The Motivation: From Uniformity to Intelligence

1.1 The Role and Limitations of Standard Replay

Experience Replay is a cornerstone of deep Q-learning, primarily for stabilizing the learning process by breaking temporal correlations in the data stream. By sampling uniformly from a large buffer of past experiences, the i.i.d. (independent and identically distributed) assumption of many gradient-based optimizers is better satisfied.

However, this uniform strategy is fundamentally inefficient from a learning perspective. It implicitly assumes all experiences hold equal value, which is demonstrably false. An agent learns most when an outcome violates its expectations.

Definition 1.1 (Learning Opportunity)**.** *The magnitude of an agent's learning opportunity from a given transition is proportional to the error in its prediction about that transition's outcome.*

In Q-learning, this prediction error is the **Temporal-Difference (TD) error**, δ_t. The TD-error measures the discrepancy between the network's prediction and the learning target:

$$\delta_t = y_t - Q(s_t, a_t; \theta)$$

where y_t is the target value (e.g., from a DDQN target). A large $|\delta_t|$ signifies a highly "surprising" event that the agent can learn much from, while a small $|\delta_t|$ signifies a predictable event that reinforces existing knowledge but offers little new insight. PER proposes to focus on these surprising events.

2 The PER Mechanism: A Two-Part Solution

Simply sampling the transitions with the highest TD-error greedily would lead to overfitting and a loss of diversity. PER introduces a more sophisticated two-part mechanism to solve this.

2.1 Part 1: Stochastic Prioritization (Controlling Greediness)

This part addresses how to sample important transitions without losing diversity.

2.1.1 Priority Assignment

The priority p_i of a transition i is defined as its absolute TD-error, with a small positive constant ϵ to ensure that no transition has zero probability of being sampled.

$$p_i = |\delta_i| + \epsilon$$

2.1.2 Stochastic Sampling

This priority is then converted into a sampling probability $P(i)$ using a hyperparameter $\alpha \in [0, 1]$:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \tag{1}$$

The exponent α controls the "greediness" of prioritization. If $\alpha = 0$, all priorities become 1, and we recover uniform random sampling. If $\alpha = 1$, sampling is directly proportional to the priority.

2.2 Part 2: Importance-Sampling (IS) Correction (Correcting Bias)

Problem 2.1 (Distribution Shift)**.** *By prioritizing, we are no longer sampling from the uniform distribution of states encountered by the agent. This introduces a bias in the sampled mini-batch. The updates from this biased sample will not converge to the true value function but to a biased one.*

To counteract this bias, we must down-weight the influence of transitions that we are now more likely to sample. This is achieved with **Importance-Sampling (IS) weights**. The weight w_i for a sample i scales its loss contribution, and is defined as:

$$w_i = \left(\frac{1}{N \cdot P(i)}\right)^\beta, \quad \text{normalized by} \quad w_i \leftarrow \frac{w_i}{\max_j(w_j)} \tag{2}$$

where N is the buffer capacity and $\beta \in [0, 1]$ is a hyperparameter that is annealed towards 1 during training.

3 Efficient Implementation: The SumTree

A naive implementation of PER would require an $O(N)$ operation for sampling. The standard solution is a binary tree data structure called a **SumTree**, which provides logarithmic time complexity ($O(\log N)$).

3.1 Python Implementation

The following is a Python implementation of the SumTree used inside our prioritized buffer.

```
import numpy as np

class SumTree:
    def __init__(self, capacity):
        self.capacity = capacity
        # The tree has 'capacity' leaf nodes. The total size is 2*capacity - 1.
        self.tree = np.zeros(2 * capacity - 1)
        # The data array stores the actual experience objects.
```

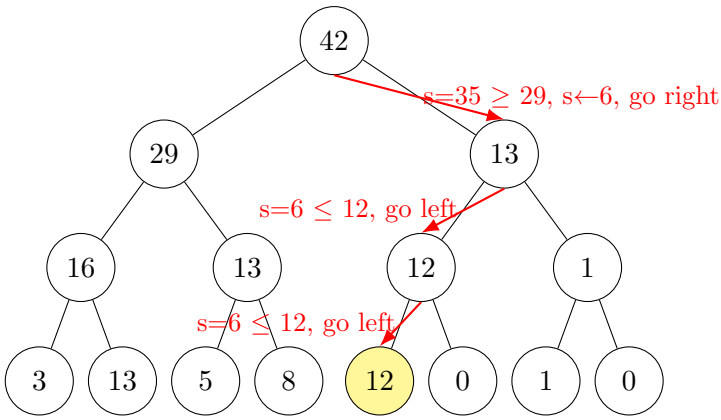

Figure 1: An illustration of the SumTree sampling process. A value s is drawn from $[0, 42]$. The tree is traversed from the root to find the corresponding leaf. In this example, a sample value of $s = 35$ correctly leads to the leaf with priority 12.

```
        self.data = np.zeros(capacity, dtype=object)
        self.data_pointer = 0 # Points to the next position to write data
        self.size = 0 # Current number of items in the buffer

    def _propagate(self, tree_index, change):
        """Propagates a change in priority up the tree."""
        parent_index = (tree_index - 1) // 2
        self.tree[parent_index] += change
        if parent_index != 0:
            self._propagate(parent_index, change)

    def _retrieve(self, tree_index, s):
        """Finds the leaf index for a given priority value 's' by traversing.
    """
        left_child_index = 2 * tree_index + 1
        right_child_index = left_child_index + 1

        if left_child_index >= len(self.tree): # Reached a leaf
            return tree_index

        if s <= self.tree[left_child_index]:
            return self._retrieve(left_child_index, s)
        else:
            return self._retrieve(right_child_index, s - self.tree[
    left_child_index])

    def total(self):
        """Returns the total priority (the value of the root)."""
```

```
        return self.tree[0]

    def add(self, priority, data):
        """Adds a new experience with a given priority."""
        # Find the leaf index to write to
        tree_index = self.data_pointer + self.capacity - 1

        self.data[self.data_pointer] = data
        self.update(tree_index, priority)

        self.data_pointer = (self.data_pointer + 1) % self.capacity
        self.size = min(self.size + 1, self.capacity)

    def update(self, tree_index, priority):
        """Updates the priority of a node and propagates the change."""
        change = priority - self.tree[tree_index]
        self.tree[tree_index] = priority
        self._propagate(tree_index, change)

    def get(self, s):
        """Gets the leaf index, priority, and data for a given value 's'."""
        leaf_index = self._retrieve(0, s)
        data_index = leaf_index - self.capacity + 1
        return (leaf_index, self.tree[leaf_index], self.data[data_index])
```

Listing 1: A Python class for the SumTree data structure.

4 Discussion

PER is a powerful component that significantly accelerates learning.

- **Hyperparameter Sensitivity:** The performance of PER is sensitive to the choice of α and β. A higher α makes the agent focus more on its errors, which can be beneficial if the TD-errors are reliable but detrimental if they are noisy. The β annealing schedule is also important for stability.
- **Synergy with Rainbow Components:** PER's true power is unlocked when combined with other improvements. DDQN provides a more stable TD-error, making the prioritization more meaningful. When combined with Distributional RL, the priority can be based on the KL-divergence between the predicted and target distributions, offering an even richer signal of "surprise" than a simple error on the mean.

By transforming the replay buffer from a simple list into a dynamic, prioritized curriculum, PER marks a critical step towards more intelligent and efficient learning agents.